

A Lexicographic Constraint-Programming Model for Open-Division Slowpitch Softball Defensive Lineups

David Russo

September 2026

Abstract

This paper specifies and evaluates a binary constraint-programming model for constructing a seven-inning defensive schedule in open-division slowpitch softball. The setting has eight to fifteen available players and up to ten simultaneous defensive positions. Unlike the associated co-ed rules profile, the Open profile imposes no gender-dependent roster or placement constraint. The principal difficulty is consequently not lineup legality alone, but the ordered reconciliation of equitable participation, stated positional preferences, and operational continuity. Explicit preferences are augmented by a deliberately asymmetric hierarchy of defensible fallback positions; pitcher and first base remain opt-in, whereas every player is eligible to catch. A lexicographic objective first minimizes playing-time spread and deviation, then minimizes non-explicit fallback assignments, then penalizes short runs, excessive positional variety, and state transitions. The Bench is modeled as a genuine continuity state, so a field-to-Bench movement is treated as a switch and consecutive bench innings are favored. The model is solved by OR-Tools CP-SAT using a nominal interactive solver-phase target and reports a conservative distinction between proven optimal and merely feasible results. For the thirteen-player Team Red instance, the implementation produces the provably fairest inning-count distribution, uses no fallback assignment, keeps every player at no more than two fielding positions, and consolidates all avoidable one-inning field and Bench runs. Automated model, mutation, application-state, and mobile-browser tests establish the principal claims. The optimizer’s returned schedule is the operational output; the application contains no subsequent captain-approval or manual-review stage.

Keywords: sports scheduling; constraint programming; CP-SAT; lexicographic optimization; playing-time fairness; human-centered operations

1 Introduction

Defensive scheduling in recreational slowpitch softball is a compact but nontrivial assignment problem. A conventional defense contains ten positions: pitcher (P), catcher (C), first base (1B), second base (2B), third base (3B), shortstop (SS), left field (LF), left center field (LC), right center field (RC), and right field (RF). A roster commonly contains more available players than defensive positions. The resulting schedule must repeatedly decide who fields, who occupies each position, and who is on the Bench.

The Open league considered here does not divide players by gender and does not impose gender minima. That simplification removes one class of hard constraints, but it does not make the practical objective obvious. Equal playing time is important; explicit positional preferences should be honored; players should not be moved unnecessarily; and a player who sits should, when possible, sit in a coherent block rather than alternate between the field and the Bench. These goals conflict. A smoother schedule is not necessarily a more preference-respecting schedule, and a perfectly continuous schedule is not necessarily fair.

The implementation resolves those conflicts by priority rather than by an opaque weighted compromise. The order is substantive: legality precedes fairness, fairness precedes explicit-position fidelity, and both

precede continuity. Lower-priority improvements are prohibited from purchasing a worse higher-priority result. This formulation is particularly appropriate for an interactive tool, where a user should not have to interpret a hidden exchange rate between one inning of inequity and one position change.

The contribution is an implemented, tested formulation with four notable features:

1. a profile-specific Open lineup rule supporting eight through fifteen available players;
2. explicit separation of stated preferences from hierarchy-derived fallback eligibility;
3. continuity modeling over both field positions and the Bench; and
4. bounded, status-aware solution stages suitable for last-minute use in a mobile web application.

2 Operational Assumptions

The model optimizes one game at a time under the following assumptions.

- A regulation planning horizon contains seven innings.
- Availability is constant throughout the game. Late arrival and early departure are outside the current input model.
- Between eight and fifteen players may be selected as available. A stored roster may be larger, but a single solve is capped at fifteen.
- Every active position is filled in every modeled inning, and one player cannot occupy two simultaneous positions.
- Gender has no legal or objective effect in the Open profile. Team Red therefore stores an internal **Unspecified** sentinel and exposes no gender control.
- A checked position is an unranked explicit preference. Derived positions are legal fallbacks, not additional preferences.
- Every player may catch. Pitcher and first base require explicit opt-in because neither is inferred from another position.
- The Bench state means only that a player is not fielding in that inning. It does not remove the player from the batting order or model an offensive substitution.
- For a given confirmed input snapshot, the returned schedule is authoritative. The designated lineup caller may alter the inputs and solve again, but no approval gate or hand edit modifies the output.

Let N denote the number of available players and K the number of active fielding positions. The Open lineup rule is fixed before optimization:

Available players N	Fielders K	Active position set
$10 \leq N \leq 15$	10	P, C, 1B, 2B, 3B, SS, LF, LC, RC, RF
$N = 9$	9	all standard positions except RF
$N = 8$	8	all standard positions except C and RF

An input with fewer than eight or more than fifteen available players is rejected before the constraint model is constructed. If ten or more players are available, the optimizer does not silently reduce the defense to nine to work around insufficient eligibility; it reports the eligibility failure.

3 Preference and Eligibility Semantics

Let the complete position universe be

$$\mathcal{S} = \{P, C, 1B, 2B, 3B, SS, LF, LC, RC, RF\},$$

and let $\mathcal{S}_K \subseteq \mathcal{S}$ be the active set determined above. For player p , let $Q_p \subseteq \mathcal{S}$ be the explicitly selected preferences. The fallback hierarchy is intentionally directional:

$$\begin{aligned} SS &\succ \{3B, 2B\}, & 3B &\succ \{2B\}, \\ LC &\succ \{LF, RC, RF\}, & LF &\succ \{RC, RF\}, \\ RC &\succ \{RF\}. \end{aligned}$$

Here $a \succ B$ means that explicitly selecting a permits assignment to members of B as fallbacks. The relation is not symmetric. It also carries no numerical distance: every inferred assignment has one unit of fallback cost. Continuity therefore may not increase the secured aggregate fallback count merely to obtain a smoother schedule. Among schedules having the same aggregate count, however, continuity may choose RF for an LC-preferring player or redistribute which player receives a fallback. The current model does not optimize individual fallback burden.

Define $h(Q_p)$ as the union of all positions directly implied by the selected positions under the displayed hierarchy. The eligible active set is

$$E_p = (Q_p \cup h(Q_p) \cup \{C\} \cup R_p) \cap \mathcal{S}_K,$$

where

$$R_p = \begin{cases} \{RC\}, & N = 8 \text{ and } RF \in Q_p, \\ \emptyset, & \text{otherwise.} \end{cases}$$

The special RF-to-RC rule applies only to the exactly eight-player defense, when both C and RF are inactive. It does not apply at nine players. Universal catcher eligibility is operational policy, while P and 1B are absent from $h(\cdot)$ and must be explicitly selected.

The fallback set is

$$F_p = E_p \setminus Q_p.$$

The model may assign p to F_p to preserve feasibility or fairness, but it minimizes such innings before evaluating any continuity term. Thus eligibility is broader than preference without erasing the semantic distinction between the two.

4 Constraint-Programming Formulation

4.1 Sets and decision variables

Let $\mathcal{P} = \{1, \dots, N\}$ be the available players and $\mathcal{I} = \{1, \dots, 7\}$ the innings. The primary binary variable is

$$x_{ips} = \begin{cases} 1, & \text{if player } p \text{ fields at position } s \text{ in inning } i, \\ 0, & \text{otherwise,} \end{cases}$$

for $i \in \mathcal{I}$, $p \in \mathcal{P}$, and $s \in E_p$. A second binary variable b_{ip} equals one exactly when player p is on the Bench in inning i . Omitting x variables for positions outside E_p makes an ineligible fielding assignment structurally impossible.

Each active position is filled exactly once:

$$\sum_{p:s \in E_p} x_{ips} = 1 \quad \forall i \in \mathcal{I}, s \in \mathcal{S}_K.$$

Each player occupies exactly one field-or-Bench state:

$$b_{ip} + \sum_{s \in E_p} x_{ips} = 1 \quad \forall i \in \mathcal{I}, p \in \mathcal{P}.$$

The latter equality simultaneously prevents double assignment and explicitly represents the Bench. Because the Open profile has no gender rule, the model contains no gender-count or gender-placement constraint.

Before these variables are constructed, a preflight procedure verifies that each active position appears in at least one E_p . Duplicate player names, blank names, invalid positions, and out-of-range player counts are likewise rejected with actionable messages.

4.2 Playing-time quantities

The number of fielding innings received by player p is

$$t_p = \sum_{i \in \mathcal{I}} \sum_{s \in E_p} x_{ips}.$$

Let

$$t^{\max} = \max_{p \in \mathcal{P}} t_p, \quad t^{\min} = \min_{p \in \mathcal{P}} t_p, \quad \Delta = t^{\max} - t^{\min}.$$

There are $H = 7K$ fielding slots. To avoid fractional expressions, define the scaled deviation from an equal share as

$$d_p = |Nt_p - H|.$$

Minimizing $\sum_p d_p$ is equivalent to minimizing total absolute deviation from H/N , up to the positive factor N .

4.3 State runs and transitions

For continuity purposes, each player has a state alphabet

$$\mathcal{R}_p = E_p \cup \{\text{Bench}\}.$$

Let w_{ipr} equal x_{ipr} for a field position $r \in E_p$ and equal b_{ip} for $r = \text{Bench}$. A run is a maximal consecutive subsequence of identical states. Consequently,

$$\text{SS} \rightarrow \text{Bench} \rightarrow \text{SS}$$

contains three runs and two transitions; it is not interpreted as one interrupted shortstop assignment.

For $i > 1$, a transition-start variable a_{ipr} represents the conjunction $w_{ipr} \wedge \neg w_{i-1,p,r}$ through

$$a_{ipr} \leq w_{ipr}, \quad a_{ipr} + w_{i-1,p,r} \leq 1, \quad a_{ipr} \geq w_{ipr} - w_{i-1,p,r}.$$

Exactly one new state starts whenever adjacent innings have different states. The first inning is not charged as a transition.

With boundary values $w_{0pr} = w_{8pr} = 0$, a one-inning-run indicator $q_{ipr}^{(1)}$ recognizes the pattern 0, 1, 0:

$$\begin{aligned} q_{ipr}^{(1)} &\leq w_{ipr}, \\ q_{ipr}^{(1)} &\leq 1 - w_{i-1,p,r}, \\ q_{ipr}^{(1)} &\leq 1 - w_{i+1,p,r}, \\ q_{ipr}^{(1)} &\geq w_{ipr} - w_{i-1,p,r} - w_{i+1,p,r}. \end{aligned}$$

Analogous four-boundary inequalities define $q_{ipr}^{(2)}$ for an exact two-inning pattern 0, 1, 1, 0. Both definitions apply to field positions and the Bench. These variables measure rather than prohibit short runs; feasibility and higher-priority fairness remain paramount.

4.4 Positional variety

Let u_{ps} equal one if player p uses field position s at least once. The standard linkage is

$$x_{ips} \leq u_{ps} \quad \forall i, p, s, \quad u_{ps} \leq \sum_i x_{ips} \quad \forall p, s.$$

Define the number of distinct field positions as

$$v_p = \sum_{s \in E_p} u_{ps}.$$

The implementation gives the bounded integer e_p the constraints

$$e_p \geq v_p - 2, \quad e_p \geq 0.$$

Minimization makes this lower bound tight for an optimized incumbent; equality to $\max(v_p - 2, 0)$ is not imposed as a primitive constraint. Bench does not contribute to v_p or e_p . The two-position target is soft: the model may exceed it to protect legality, fairness, or preference fidelity.

5 Lexicographic Objective

The optimization priorities are, in strict order:

1. minimize playing-time spread Δ ;
2. minimize total scaled deviation $\sum_p d_p$;
3. minimize fallback innings $f = \sum_{i,p} \sum_{s \in F_p} x_{ips}$;
4. minimize one-inning state runs $o_1 = \sum_{i,p,r} q_{ipr}^{(1)}$;
5. minimize excess field positions $e = \sum_p e_p$;

6. minimize exact two-inning state runs $o_2 = \sum_{i,p,r} q_{ipr}^{(2)}$;
7. minimize total distinct field positions $v = \sum_p v_p$; and
8. minimize adjacent state transitions $a = \sum_{i,p,r} a_{ipr}$.

The first two priorities are combined without ambiguity. If D^{UB} is an upper bound on $\sum_p d_p$, the fairness objective is

$$J_F = (D^{UB} + 1)\Delta + \sum_p d_p.$$

The multiplier ensures that any one-unit reduction in spread dominates every possible change in total deviation.

After fairness and fallback counts have been secured, the continuity objective is

$$J_C = c_1 o_1 + c_e e + c_2 o_2 + c_v v + a,$$

where each coefficient is constructed to exceed the maximum possible total contribution of all lower-order terms. In implementation terms,

$$c_v > a^{UB}, \quad c_2 > c_v v^{UB} + a^{UB}, \quad c_e > c_2 o_2^{UB} + c_v v^{UB} + a^{UB},$$

and c_1 analogously dominates the entire remaining expression. Hence J_C is a compact encoding of the stated lexicographic order rather than a subjective weighted average.

Three consequences deserve emphasis. First, continuity cannot increase the aggregate fallback count secured by the preference phase, although it may choose among or redistribute fallbacks at that same count. Second, a Bench run is evaluated by the same continuity logic as a field-position run. Third, minimizing two-inning runs occurs only after one-inning runs and excess positions, so the solver prefers three-or-more-inning blocks when higher priorities permit but does not sacrifice an avoidable extra field position merely to lengthen a run.

6 Bounded Solution Procedure

The model is implemented with the OR-Tools CP-SAT solver, which supports Boolean and integer constraint models and distinguishes statuses including *OPTIMAL*, *FEASIBLE*, and *INFEASIBLE* [1]. The interactive default is a nominal five-second multi-phase solve target measured only after model construction. It is not a hard end-to-end wall-clock bound: model-building time, Python and Streamlit overhead, CP-SAT time-limit behavior, and the implementation’s minimum per-phase and subphase allocations of 0.05–0.1 seconds can make observed elapsed time exceed five seconds. The implementation uses the following nominally bounded phases:

1. **Arithmetic fairness certificate.** Add $\lfloor H/N \rfloor \leq t_p \leq \lceil H/N \rceil$ for every player and test feasibility. A feasible assignment proves that the arithmetic lower bounds for spread and scaled absolute deviation are attainable subject to all hard constraints and therefore globally optimal.
2. **General fairness search.** If that slice cannot be established, minimize J_F within the designated fairness budget.
3. **Preference phase.** Fix the achieved spread and deviation, minimize f , and fix the exact fallback incumbent before any continuity phase. If optimality is not proved, the incumbent is still frozen rather than silently relaxed.
4. **One-inning phase.** Test the ideal lower bound in which the only single-inning field run is forced by

$t_p = 1$ and the only single-inning Bench run is forced by $t_p = 6$. Within this hard slice, minimize e as look-ahead to the next tier. If the ideal slice is not found, minimize $o_1(E^{UB} + 1) + e$, where E^{UB} bounds total excess positions, and preserve the one-inning incumbent. The coefficient keeps every unit of o_1 strictly more important than all possible changes in e .

5. **Two-position phase.** If look-ahead proves the value of e or reaches its analytic lower bound of zero, fix that value immediately. Otherwise, first test the $e = 0$ slice inside the existing phase budget, then minimize e with the unspent remainder. A proved value is fixed; an unproved incumbent becomes a non-worsening upper bound.
6. **Continuity phase.** Minimize J_C for the remaining nominal phase budget, using prior assignments as solution hints where applicable. Because a hint is not a quality guarantee, constrain J_C to be no larger than the incumbent value and accept a timed candidate only when its (o_1, e, o_2, v, q) vector is lexicographically no worse than that incumbent.

Each accepted phase returns a legal assignment. Higher-priority values are constrained before lower-priority search begins. A final status of *OPTIMAL* is exposed only if fairness, fallback use, one-inning runs, excess positions, and the final continuity objective are all proven. Any legal incumbent lacking one of those proofs is truthfully labeled *FEASIBLE*. The latter label concerns proof of objective optimality, not legality.

The web interface reports the active phase while a solve is in progress and shows a small animated stick figure. This deliberately low-technology cue serves a serious interaction purpose: mobile users receive immediate evidence that their button press was accepted and that the application is computing, rather than mistaking a multi-second solve for a dropped tap.

7 Team Red Case Study

The checked-in Team Red template contains thirteen players, all initially available under locked Open rules. Its explicit preferences are:

Player	Explicit preferences	Player	Explicit preferences
David R	2B, RC	Justin	1B, RC
Andrew	C, 2B, RF	Kevin	2B, LC
David	RF	Dung	P
Heather	LF, LC	Ryan	1B, 3B
Tristan	C, 3B	Brad	C, LF
AP	SS, LF	Chris	C, 2B, RF
Marty	SS, LC		

With $N = 13$ and $K = 10$, the instance contains $H = 70$ fielding slots. Dung is the only explicitly eligible pitcher and therefore must pitch all seven innings. A playing-time spread of one is impossible: if a seven-inning player were present, all thirteen players would need at least six innings, requiring at least 78 slots. A schedule with one player at seven innings, three at six, and nine at five uses exactly

$$1(7) + 3(6) + 9(5) = 70$$

slots and attains spread two. It therefore proves the minimum spread for this instance. To establish the second fairness tier, let a , b , and c be the numbers of players receiving five, six, and seven innings. Then

$a + b + c = 13$, $5a + 6b + 7c = 70$, and hence $b + 2c = 5$. Dung forces $c \geq 1$. The only possibilities are $(a, b, c) = (9, 3, 1)$ and $(10, 1, 2)$, with scaled deviations 90 and 100 respectively. The first distribution is therefore optimal for both fairness tiers, and its deviation is

$$1|13(7) - 70| + 3|13(6) - 70| + 9|13(5) - 70| = 90.$$

The regression instance further establishes the following operational properties:

- all seven innings fill all ten positions with distinct players;
- Dung occupies P in every inning;
- all assignments are explicit preferences, so $f = 0$;
- every player uses at most two field positions;
- every field-position run has length at least two;
- each five-inning player’s two Bench innings are consecutive;
- the only three one-inning runs are the mathematically unavoidable Bench runs of the three six-inning players;
- the schedule contains 15 exact two-inning state runs, 18 adjacent state transitions, and 19 total player-position incidences.

These measurements characterize the tested objective outcome, not a unique name-by-name lineup. Multiple schedules can share the same lexicographic quality. The application returns one solver-selected representative and uses it directly as the final defensive schedule.

The instance also exposes availability risk clearly. If Dung is marked unavailable, no remaining player has opted into P. With eleven available players, simultaneous absences by Justin and Ryan remove all 1B coverage, while simultaneous absences by AP and Marty remove all SS coverage. Adequate headcount therefore does not guarantee positional feasibility. The correct response is an infeasibility message identifying the uncovered position, not an invented assignment and not a reduced nine-player lineup. By contrast, a missing explicit catcher does not create infeasibility because every player is eligible to catch by policy.

8 Verification and Falsification Strategy

Validation is organized around properties rather than visual plausibility. At the September 2026 release gate, 100 Python tests and 14 Playwright browser tests pass. The browser suite runs each workflow in a mobile-touch project and a desktop-mouse project. The principal evidence includes:

1. **Legality invariants:** seven innings, exact active-position coverage, no duplicated player within an inning, valid eligibility, and reported summary counts reconstructed independently from assignments.
2. **Open-profile boundaries:** legal eight-, nine-, and ten- fielder position sets; no gender error in Open; no nine-player RF-to-RC special promotion; universal C fallback; and explicit P/1B requirements.
3. **Team Red contract:** exact names and preferences, internal **Unspecified** gender, locked Open rules, all players initially available, fair inning distribution, and the continuity metrics reported above.
4. **Preference-priority witness:** the original fixture has a solver-proven seven-fallback optimum with continuity vector $(5, 1, 16, 20, 21)$. Promoting Third Specialist’s already-eligible 2B from a fallback to an explicit preference leaves the eligibility sets unchanged and yields the smoother vector $(5, 0, 14, 20, 20)$. Rescored under the original preferences, that exhibited schedule uses nine fallbacks. The five vector components are, respectively, one-inning runs, excess positions, two-inning runs, total distinct positions, and adjacent transitions; the first improvement is excess positions, from one to

zero. The original optimizer nevertheless selects the seven-fallback result, proving that continuity cannot buy a worse aggregate fallback count.

5. **Mutation test:** weakening the exact fallback-incumbent freeze to a one-sided inequality is detected. The regression inspects the model domain and requires the exact singleton interval $[f, f]$.
6. **Interaction tests:** real individual position taps, complete name entry, gender change where applicable, save/reopen persistence, optimization, the complete semantic seven-inning matrix, CSV download, invalid optimization recovery, removal, reset, and independent browser sessions.

The strict preference witness is important because a test that merely observes zero fallback use is vacuous: continuity cannot demonstrate an illegal trade when no trade is available. The constructed counterexample supplies a smoother but preference-worse alternative and verifies that the implemented priority rejects it. This is closer to falsification than to screenshot confirmation.

9 Human and Operational Interpretation

The objective order encodes several practical slowpitch considerations. Continuous field-position blocks reduce repeated instruction, warm-up changes, and uncertainty between innings. Modeling Bench as a state prevents a schedule from appearing stable while repeatedly sending the same player on and off the field. Consecutive Bench innings also make the rotation easier to communicate from a phone in a dugout.

The hierarchy supplies controlled resilience without pretending all positions are interchangeable. Its SS-to-3B/2B, 3B-to-2B, and LC-to-LF/RC/RF pathways are declared team-policy eligibility rules, not universal claims that one position is easier than another. Third base remains the hot corner, and center-and corner-field reads can demand different skills. The separate fallback phase prevents continuity from increasing the secured aggregate fallback count, while still permitting redistribution among schedules tied at that count. Pitching and first base remain special, explicit choices.

Operational authority is scoped to an input snapshot, not to every open web session. Team Red’s defaults mark all thirteen players available, but that is not attendance confirmation. Before solving, one designated lineup caller must mark every scratch. Any later availability or preference change requires a new solve, and the newly displayed or downloaded matrix supersedes the previous one without hand editing. Because browser sessions are independent and not synchronized, the team should distribute the result from that single source, preferably by downloading the final matrix before the first pitch.

No captain-review stage follows optimization. Such a stage would blur the contract by presenting a computed schedule and then asking a human to repair it without rechecking legality, fairness, or continuity. In the implemented workflow, changed judgments are expressed as changed availability or preferences and submitted to a new solve. The optimizer, rather than an unvalidated hand edit, remains the source of the displayed and downloaded lineup.

10 Limitations and Threats to Validity

The model is intentionally narrow. Bench means not fielding; it does not alter batting order. The model does not optimize offensive substitutions, opponent tendencies, score-dependent tactics, player fatigue, or inning-specific availability. It balances playing time within one game and does not carry season-to-date equity between sessions. A shortened game simply leaves later rows unused and may make realized playing time less equal than the planned seven-inning distribution; extra innings are outside the fixed horizon. The hierarchy is a declared operational policy, not an empirical estimate of every individual’s

defensive ability.

Universal catcher eligibility is likewise a policy assumption. The software correctly applies it to every roster and does not offer a catcher opt-out. If a future league or team rejects that assumption, the input schema and hard eligibility rules would need an explicit change; silently weakening the rule inside the solver would be inappropriate.

The objective treats all explicit preferences as equal and all inferred fallbacks as equal. It does not know that one player strongly favors LC over LF unless the roster is edited to reflect only the intended choices. A future ranked-preference model could add tiers, but it would require additional input and a revised priority contract.

Finally, each CP-SAT phase is assigned a nominal time limit. A *FEASIBLE* result is guaranteed to satisfy the encoded hard constraints but may lack a proof that every reported objective tier is globally best. Conservative status reporting makes this uncertainty visible, and the web interface reports observed elapsed time. This paper does not infer latency percentiles or a hard wall-clock guarantee from the mere existence of a benchmark utility. Any expansion beyond the eight-to-fifteen-player scope or seven-inning horizon requires renewed, reproducible timing measurement.

11 Conclusion

Open-division defensive scheduling is best viewed as a lexicographic state- assignment problem. The model fills every required position, equalizes playing time as far as eligibility permits, preserves explicit preferences before using a controlled hierarchy, and then constructs long, low-switch field and Bench runs. The distinction between explicit preference and inferred eligibility is essential: flexibility is available to obtain a legal, fair schedule, but the aggregate use of that flexibility is secured before the solver makes its continuity statistics prettier. Individual fallback assignments may still move among schedules tied at the same aggregate count.

The Team Red case demonstrates the formulation on a real thirteen-player roster. Its tested schedule achieves the minimum possible playing-time spread and scaled deviation, uses no fallback inning, respects the two-position target, and consolidates every avoidable one-inning state. The staged CP-SAT implementation is designed for game-day interaction, reports observed elapsed time and proof status honestly, and is exercised through both model-level and real-browser regressions. The final lineup is consequently not a suggestion awaiting captain repair; it is the authoritative output of the stated rules, confirmed input snapshot, and priorities.

A Algorithmic Summary

The implemented procedure may be summarized as follows:

1. normalize and validate the selected available players;
2. select the Open active-position set from N ;
3. derive E_p and F_p while preserving Q_p ;
4. preflight active-position coverage;
5. construct field, Bench, fairness, run, position-use, and transition variables;
6. attempt the arithmetic fairness certificate, otherwise optimize fairness;
7. constrain the achieved fairness pair exactly;
8. minimize and freeze fallback innings;

9. minimize and preserve one-inning state runs;
10. minimize excess positions and prevent subsequent worsening;
11. minimize the lexicographically weighted continuity expression;
12. translate the legal incumbent into an inning matrix and player summary; and
13. label the result *OPTIMAL* only when all required proofs exist, otherwise label the legal result *FEASIBLE*.

B Application and Reproducibility Artifacts

The public neutral application is available at <https://softball-fielding-optimizer.streamlit.app/>. It presents an explicit first-load choice among Here For The Beer, Team Red, and a blank roster. Team Red loads the roster in `team_red_roster.csv` under locked Open rules. The existing Here For The Beer deployment remains available at <https://here-for-the-beer.streamlit.app/> and retains its Co-ed defaults.

The implementation, normative specification, test suite, and benchmark utilities are maintained in the public repository at <https://github.com/davidluizrusso/softball-fielding>. Streamlit Community Cloud deploys an application from a GitHub repository and entrypoint and provides a shareable `streamlit.app` URL [3]. Python dependencies are declared in `requirements.txt`, consistent with the platform’s dependency mechanism [4].

The command-line verification entry points are:

- `python -m pytest` for model and Streamlit state tests;
- `npm run test:browser` for mobile-touch and desktop-mouse Playwright workflows;
- `python scripts/benchmark_fairness.py -budgets 1 2 3` for repeated fairness-phase stress trials at selected roster sizes; and
- `python scripts/benchmark_hftb.py`
`-processes 4 -runs-per-process 5 -budget 5`
`-output benchmarks/hftb-5s-reference.json` for the opt-in 20-run HFTB latency, phase-timing, and lexicographic-quality contract across fresh processes.

References

- [1] Google for Developers. *CP-SAT Solver: OR-Tools*. Accessed September 8, 2026.
- [2] Google OR-Tools. *OR-Tools Optimization Software*. Accessed September 8, 2026.
- [3] Streamlit. *Deploy Your App on Community Cloud*. Accessed September 8, 2026.
- [4] Streamlit. *App Dependencies for Your Community Cloud App*. Accessed September 8, 2026.
- [5] D. Bertsimas and J. N. Tsitsiklis. *Introduction to Linear Optimization*. Athena Scientific, 1997.
- [6] H. P. Williams. *Model Building in Mathematical Programming*, 5th ed. Wiley, 2013.